SCHOOLOFCOMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENTOFCOMPUTER SCIENCE ENGINEERING	
ProgramName:B. Tech		**AssignmentType: Lab**	**AcademicYear:**2025-2026
CourseCoordinatorName		Venkataramana Veeramsetty	
Instructor(s)Name		Dr. V. Venkataramana (Co-ordinator) Dr. T. Sampath Kumar Dr. Pramoda Patro Dr. Brij Kishor Tiwari Dr.J.Ravichander Dr. Mohammand Ali Shaik Dr. Anirodh Kumar Mr. S.Naresh Kumar Dr. RAJESH VELPULA Mr. Kundhan Kumar Ms. Ch.Rajitha Mr. M Prakash Mr. B.Raju Intern 1 (Dharma teja) Intern 2 (Sai Prasad) Intern 3 (Sowmya) NS_2 (Mounika)	
CourseCode	24CS002PC215	**CourseTitle**	AI Assisted Coding
Year/Sem	II/I	**Regulation**	R24
DateandDay of Assignment	Week8 - WednesDay	**Time(s)**	
Duration	2 Hours	**Applicableto Batches**	
AssignmentNumber:16.3(Presentassignmentnumber)/**24**(Totalnumberofassignments)			

Q.No.	Question	*ExpectedTime to complete*
1	**Lab 16 – Database Design and Queries: Schema Design and SQL Generation** **Lab Objectives** • To practice basic SQL query generation with AI assistance. • To analyze AI-suggested queries for correctness and efficiency.	Week5 - Monday

- To understand how AI can help in documenting and improving database logic.

Learning Outcomes

After completing this lab, students will be able to:

1. Use AI tools to design a simple ER diagram / schema for a given scenario.
2. Generate CREATE TABLE statements using AI.
3. Write and refine basic SQL queries (SELECT, INSERT, UPDATE, DELETE).
4. Validate correctness and efficiency of AI-generated SQL.
5. Compare AI-generated vs manually written queries.

Task Description #1 – Schema Generation

Task: Ask AI to design a schema for a Library Management System (Tables: Books, Members, Loans).

SQL Code

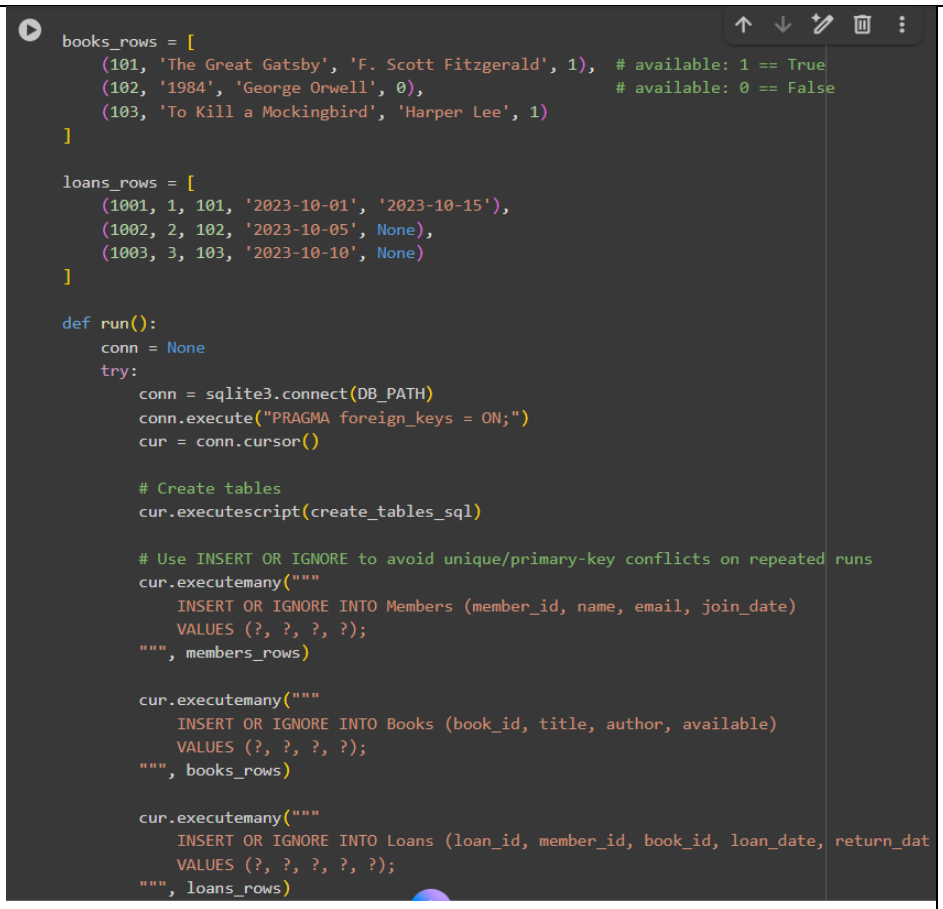
```
CREATE TABLE Members (
    member_id INT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100) UNIQUE,
    join_date DATE
);

CREATE TABLE Books (
    book_id INT PRIMARY KEY,
    title VARCHAR(200),
    author VARCHAR(100),
    available BOOLEAN
);

CREATE TABLE Loans (
    loan_id INT PRIMARY KEY,
    member_id INT,
    book_id INT,
    loan_date DATE,
    return_date DATE,
    FOREIGN KEY (member_id) REFERENCES Members(member_id),
    FOREIGN KEY (book_id) REFERENCES Books(book_id)
);
```

Task Description #2 – Error Insert Data

Task: Ask AI to generate INSERT INTO queries for the schema above (3 sample records per table).

ANSWER;

```
import sqlite3
from sqlite3 import Error

DB_PATH = "library.db"

create_tables_sql = """
PRAGMA foreign_keys = ON;

CREATE TABLE IF NOT EXISTS Members (
    member_id INTEGER PRIMARY KEY,
    name TEXT,
    email TEXT UNIQUE,
    join_date DATE
);

CREATE TABLE IF NOT EXISTS Books (
    book_id INTEGER PRIMARY KEY,
    title TEXT,
    author TEXT,
    available INTEGER
);

CREATE TABLE IF NOT EXISTS Loans (
    loan_id INTEGER PRIMARY KEY,
    member_id INTEGER,
    book_id INTEGER,
    loan_date DATE,
    return_date DATE,
    FOREIGN KEY (member_id) REFERENCES Members(member_id),
    FOREIGN KEY (book_id) REFERENCES Books(book_id)
);
"""

members_rows = [
    (1, 'Alice Smith', 'alice@example.com', '2023-05-01'),
    (2, 'Bob Johnson', 'bob@example.com', '2023-06-15'),
    (3, 'Charlie Brown', 'charlie@example.com', '2023-07-20')
```



```
books_rows = [
    (101, 'The Great Gatsby', 'F. Scott Fitzgerald', 1),  # available: 1 == True
    (102, '1984', 'George Orwell', 0),                    # available: 0 == False
    (103, 'To Kill a Mockingbird', 'Harper Lee', 1)
]

loans_rows = [
    (1001, 1, 101, '2023-10-01', '2023-10-15'),
    (1002, 2, 102, '2023-10-05', None),
    (1003, 3, 103, '2023-10-10', None)
]

def run():
    conn = None
    try:
        conn = sqlite3.connect(DB_PATH)
        conn.execute("PRAGMA foreign_keys = ON;")
        cur = conn.cursor()

        # Create tables
        cur.executescript(create_tables_sql)

        # Use INSERT OR IGNORE to avoid unique/primary-key conflicts on repeated runs
        cur.executemany("""
            INSERT OR IGNORE INTO Members (member_id, name, email, join_date)
            VALUES (?, ?, ?, ?);
        """, members_rows)

        cur.executemany("""
            INSERT OR IGNORE INTO Books (book_id, title, author, available)
            VALUES (?, ?, ?, ?);
        """, books_rows)

        cur.executemany("""
            INSERT OR IGNORE INTO Loans (loan_id, member_id, book_id, loan_date, return_dat
            VALUES (?, ?, ?, ?, ?);
        """, loans_rows)
```

```
            VALUES (?, ?, ?, ?);
        """, members_rows)

        cur.executemany("""
            INSERT OR IGNORE INTO Books (book_id, title, author, available)
            VALUES (?, ?, ?, ?);
        """, books_rows)

        cur.executemany("""
            INSERT OR IGNORE INTO Loans (loan_id, member_id, book_id, loan_date, return_dat
            VALUES (?, ?, ?, ?, ?);
        """, loans_rows)

        conn.commit()

        # Fetch and display rows to confirm
        print("\nMembers:")
        for row in cur.execute("SELECT * FROM Members ORDER BY member_id;"):
            print(row)

        print("\nBooks:")
        for row in cur.execute("SELECT * FROM Books ORDER BY book_id;"):
            print(row)

        print("\nLoans:")
        for row in cur.execute("SELECT * FROM Loans ORDER BY loan_id;"):
            print(row)

        print("\n✅ Done — tables created/updated and sample records inserted (or ignored i

    except Error as e:
        print("❌ SQLite error:", e)
    finally:
        if conn:
            conn.close()

if __name__ == "__main__":
    run()
```

```
Members:
(1, 'Alice Smith', 'alice@example.com', '2023-05-01')
(2, 'Bob Johnson', 'bob@example.com', '2023-06-15')
(3, 'Charlie Brown', 'charlie@example.com', '2023-07-20')

Books:
(101, 'The Great Gatsby', 'F. Scott Fitzgerald', 1)
(102, '1984', 'George Orwell', 0)
(103, 'To Kill a Mockingbird', 'Harper Lee', 1)

Loans:
(1001, 1, 101, '2023-10-01', '2023-10-15')
(1002, 2, 102, '2023-10-05', None)
(1003, 3, 103, '2023-10-10', None)

✅ Done — tables created/updated and sample records inserted (or ignored if already present)
```

Task Description #3 – Basic Queries

Task: Use AI to generate a query to list all books borrowed by a specific member

ANSWERS;

```python
import sqlite3
from sqlite3 import Error

DB_PATH = "library.db"

def list_borrowed_books(member_id):
    """Lists all books borrowed by a specific member."""
    conn = None
    try:
        conn = sqlite3.connect(DB_PATH)
        cur = conn.cursor()

        cur.execute("""
            SELECT
                b.title,
                b.author,
                l.loan_date,
                l.return_date
            FROM Loans l
            JOIN Books b ON l.book_id = b.book_id
            WHERE l.member_id = ?;
        """, (member_id,))

        rows = cur.fetchall()

        if rows:
            print(f"\nBooks borrowed by Member ID {member_id}:")
            for row in rows:
                print(f"- Title: {row[0]}, Author: {row[1]}, Loan Date: {row[2]}, Return Da
        else:
            print(f"\nMember ID {member_id} has not borrowed any books.")

    except Error as e:
        print("❌ SQLite error:", e)
    finally:
        if conn:
```

```python
            , (member_id,))

        rows = cur.fetchall()

        if rows:
            print(f"\nBooks borrowed by Member ID {member_id}:")
            for row in rows:
                print(f"- Title: {row[0]}, Author: {row[1]}, Loan Date: {row[2]}, Return Da
        else:
            print(f"\nMember ID {member_id} has not borrowed any books.")

    except Error as e:
        print("❌ SQLite error:", e)
    finally:
        if conn:
            conn.close()

# Example usage: List books borrowed by Member ID 1
list_borrowed_books(1)

# Example usage: List books borrowed by Member ID 2
list_borrowed_books(2)
```

```
Books borrowed by Member ID 1:
- Title: The Great Gatsby, Author: F. Scott Fitzgerald, Loan Date: 2023-10-01, Return Date:

Books borrowed by Member ID 2:
- Title: 1984, Author: George Orwell, Loan Date: 2023-10-05, Return Date: None
```

Task Description #4 – Update and Delete Queries

Task: Generate queries with AI for:

- Updating a book's availability to FALSE when borrowed.

```python
import sqlite3
from sqlite3 import Error

DB_PATH = "library.db"

def update_book_availability(book_id, available):
    """Updates the availability status of a book."""
    conn = None
    try:
        conn = sqlite3.connect(DB_PATH)
        cur = conn.cursor()

        cur.execute("""
            UPDATE Books
            SET available = ?
            WHERE book_id = ?;
        """, (available, book_id))

        conn.commit()
        print(f"✅ Book ID {book_id} availability updated to {bool(available)}.")

    except Error as e:
        print("❌ SQLite error:", e)
    finally:
        if conn:
            conn.close()

# Example usage: Mark book ID 101 as unavailable (borrowed)
update_book_availability(101, 0)

# Example usage: Mark book ID 102 as available (returned)
update_book_availability(102, 1)
```

```
✅ Book ID 101 availability updated to False.
✅ Book ID 102 availability updated to True.
```

- Deleting a member record safely.

```python
import sqlite3
from sqlite3 import Error

DB_PATH = "library.db"

def delete_member(member_id):
    """Deletes a member record and their associated loans."""
    conn = None
    try:
        conn = sqlite3.connect(DB_PATH)
        cur = conn.cursor()

        # Delete associated loan records first due to foreign key
        cur.execute("""
            DELETE FROM Loans
            WHERE member_id = ?;
        """, (member_id,))

        # Then delete the member record
        cur.execute("""
            DELETE FROM Members
            WHERE member_id = ?;
        """, (member_id,))

        conn.commit()
        print(f"✅ Member ID {member_id} and associated loans dele

    except Error as e:
        print("❌ SQLite error:", e)
    finally:
        if conn:
            conn.close()

# Example usage: Delete Member ID 3
delete_member(3)
```

```
✅ Member ID 3 and associated loans deleted.
```